

Programa de Ingeniería de Sistemas y Computación

Universidad del Quindío

Curso: Programación Avanzada

Guía: 12

Título: IMPLEMENTACIÓN DE SEGURIDAD CON JWT EN SPRING BOOT

Duración estimada: 240 minutos

Docentes: Christian Andrés Candela

IMPLEMENTACIÓN DE SEGURIDAD CON JWT EN SPRING BOOT

OBJETIVO

Comprender los fundamentos de seguridad en aplicaciones empresariales Spring Boot, implementar autenticación y autorización usando **JWT (JSON Web Tokens)** a través de **Spring Security 7+**, persistir credenciales de usuario seguras estructurando en Base de Datos Relacional (H2/JPA), y asentar las bases para proteger las APIs RESTful expuestas hasta el momento.

CONCEPTOS BÁSICOS

Autenticación: Proceso de verificar y validar *QUIÉN* eres dentro del ecosistema tecnológico (Ej: login con correo/contraseña contra una DataBase H2).

Autorización: Determinar *QUÉ* recursos o URL puedes visitar y operar una vez autenticado (Rol ADMIN vs Rol USER).

Spring Security (Lambda DSL): Framework robusto estandarizado que protege el ciclo HTTP usando una Flujo de Cadenas de Filtros (SecurityFilterChain).

Filtros (Filters): Centinelas dentro de Tomcat que interceptan anticipadamente todas las solicitudes.

JWT (JSON Web Token): Estándar para transmitir información blindada de lectura libre mediante Hash.

Access Token: Pasaporte de corta duración con el que el usuario accede a operaciones

protegidas desde el front-end a los MS.

Password Encoding (Bcrypt): Estrategia de encriptación pesada obligatoria para persistir claves en nuestra base de datos.

UserDetails : La Interfaz de contrato con la cual Spring reconoce y procesa la identidad de un registro nuestro en base de datos.

CONTEXTUALIZACIÓN TEÓRICA

Seguridad, Autenticación y Autorización

En el desarrollo de aplicaciones modernas, la seguridad es un aspecto fundamental para garantizar la protección de los datos y la integridad del sistema. Los conceptos clave en este ámbito son:

1. Autenticación:

Es el proceso mediante el cual se verifica la identidad de un usuario o sistema. Por ejemplo, cuando un usuario inicia sesión en una aplicación utilizando su nombre de usuario y contraseña, la aplicación valida que esa combinación sea correcta. La autenticación asegura que solo los usuarios legítimos puedan acceder al sistema.

2. Autorización:

Una vez que un usuario ha sido autenticado, la autorización determina qué recursos puede acceder y qué acciones puede realizar dentro del sistema. Por ejemplo, un administrador puede tener acceso a todas las funcionalidades de una aplicación, mientras que un usuario estándar solo puede acceder a ciertas áreas limitadas.

3. Mecanismos de Implementación:

Existen varios mecanismos para implementar la autenticación y autorización en aplicaciones web:

Cookies y Sesiones: Tradicionalmente utilizados en aplicaciones web, donde el servidor almacena información de sesión y el cliente guarda una cookie para identificarse.

OAuth2: Un protocolo ampliamente utilizado para autenticación y autorización, especialmente en aplicaciones distribuidas y APIs.

JWT (JSON Web Tokens): Un estándar moderno y ligero para transmitir información segura entre partes como objetos JSON. Es ideal para APIs RESTful y microservicios debido a su naturaleza stateless (sin estado).

JWT (JSON Web Token)

JWT es un estándar abierto ([RFC 7519](#)) que permite transmitir información entre dos partes de manera segura y compacta.

Anatomía de un JWT

Un JSON Web Token consta de tres componentes codificados en Base64URL, separados por puntos:

```
[Header].[Payload].[Signature]
```

a. Header

Contiene metadatos sobre el tipo de token y algoritmo de firma:

```
{  
  "alg": "HS512",  
  "typ": "JWT"  
}
```

b. Payload (Claims)

Incluye los "reclamos" o datos del usuario y sistema:

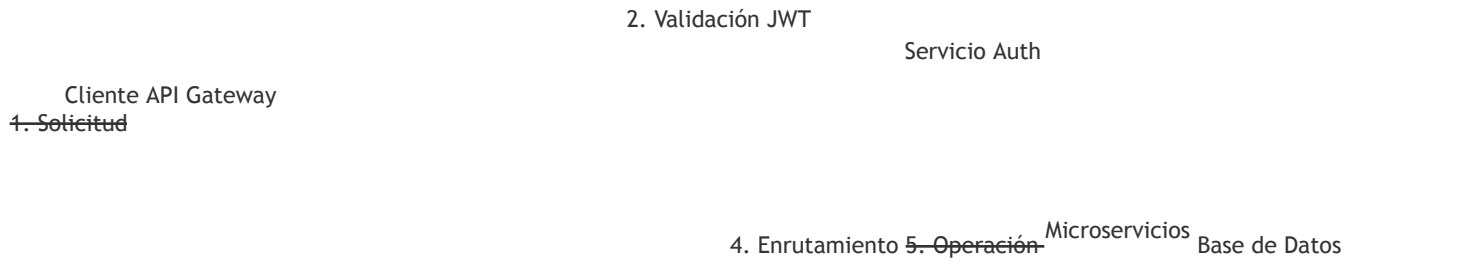
```
{  
  "sub": "usuario@dominio.com",  
  "iss": "servicio-autenticador",  
  "exp": 1735689600,  
  "roles": ["ADMIN", "USER"]  
}
```

c. Signature

Firma digital que garantiza la integridad del token:

```
HMACSHA512(  
  base64UrlEncode(header) + "." + base64UrlEncode(payload),  
  secret-key  
)
```

Modelo de Seguridad en Arquitecturas Modernas



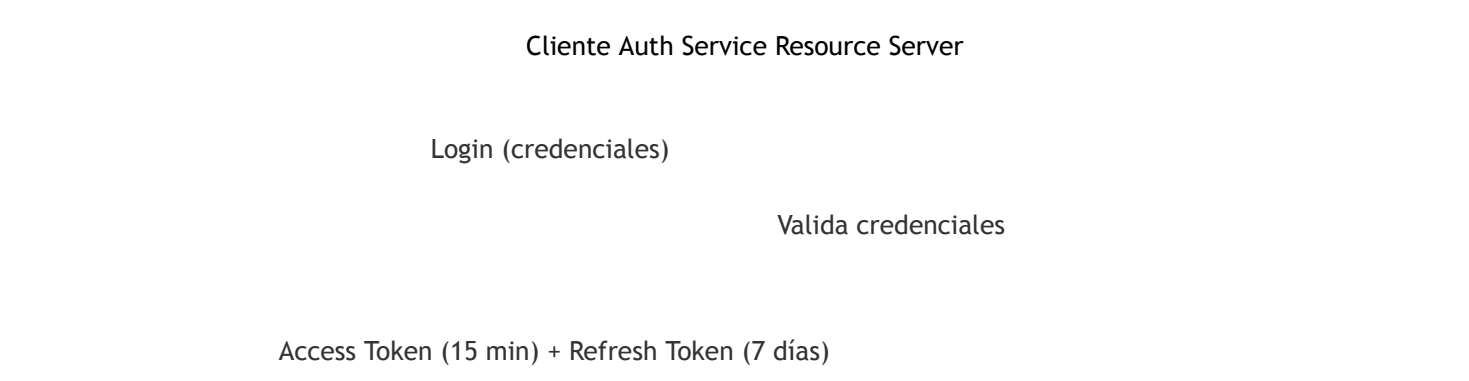
Características Clave:

- Stateless:** Elimina necesidad de sesiones almacenadas en servidor
- Descentralizado:** Cada servicio puede validar tokens independientemente
- Escalable:** Ideal para arquitecturas de microservicios

Algoritmos de Firma: HMAC vs RSA

Parámetro	HMAC (Simétrico)	RSA (Asimétrico)
Velocidad	Más rápido (operaciones hash)	Más lento (criptografía PKI)
Seguridad	Clave secreta compartida	Par clave pública/privada
Casos de Uso	Entornos monolíticos	Microservicios distribuidos
Ejemplo	HS512	RS512

Ciclo de Vida del Token





Cliente Auth Service Resource Server

Amenazas y Mitigaciones

Ataque	Descripción	Contramedida JWT
Token Theft	Robo de token en tránsito	Usar HTTPS + Cookies HttpOnly
Algorithm Swap	Cambiar algoritmo a "none"	Validar explícitamente el algoritmo
Token Replay	Reutilización de tokens robados	Tiempos de vida cortos + Listas de revocación

Ataque	Descripción	Contramedida JWT
XSS	Inyección de scripts maliciosos	Sanitizar inputs + CSP headers

Integración con OAuth2.0

JWT funciona como formato estándar para tokens en OAuth2:

1. Authorization Code

Cliente ~~3. Acceso API~~ Resource Server ~~4. Valida JWT~~ Auth Server

2. JWT Access Token

Ventajas:

Tokens auto-contenidos sin necesidad de consultar al servidor

Compatibilidad con flujos: Authorization Code, Client Credentials, Implicit

Limitaciones y Consideraciones

1. Tamaño de Token:

Un JWT típico ocupa ~1KB (problema en móviles con redes lentas)

2. Revocación:

Dificultad para invalidar tokens antes de su expiración

Soluciones:

Listas de revocación cortas (denylist)

Tiempos de vida ultracortos (30s-1min) con refresh tokens

3. Almacenamiento en Cliente:

Opción Segura: Cookies con flags HttpOnly y Secure

Riesgoso: LocalStorage/SessionStorage (vulnerable a XSS)

Casos de Uso Ideales

1. APIs REST/GraphQL

2. Single Page Applications (SPA)

3. Comunicación entre Microservicios

4. Autenticación Móvil (Apps nativas)

5. Integración con Terceros (Webhooks)

Evolución del Estándar

Versión	Año	Novedades
JWT	2015	Primer RFC (7519)
JWS	2015	Especificación de firma (7515)
JWE	2016	Estándar de encriptación (7516)
JWT-bis	2023	Actualizaciones de seguridad y formato

Comparativa con Alternativas

Tecnología	Ventajas	Desventajas
Session Cookies	Sencillez, revocación fácil	Stateful, problemas en escalado
Opaque Tokens	Revocación centralizada	Requieren almacenamiento
SAML	Estándar empresarial	Complejidad XML/SOAP
JWT	Portabilidad, stateless	Gestión manual de seguridad

BUENAS PRÁCTICAS

1. Nunca Almacenes Secretos en Código

Usa variables de entorno o servicios como Vault para claves JWT.

2. Usa HTTPS

Siempre en producción para prevenir ataques MITM.

3. Manejo Seguro de Refresh Tokens

Almacénalos en HttpOnly cookies con flags Secure y SameSite.

4. Validación Estricta de Tokens

Valide adecuadamente su token con el fin de evitar ataques comunes.

5. Limitación de Tiempo de Vida

Access Token: 15-30 minutos

Refresh Token: 7-30 días

JWT vs Base de Datos en Microservicios

Tradicionalmente (Web 1.0 monolítica) se creaba un objeto `HTTPSession` enorme en la RAM del servidor. Hoy, Spring en su estándar para RESTful opta por un paradigma `Stateless` (sin estado de memoria). Tu JWT viaja de id a id sin que el microservicio recuerde que existes hasta desencriptar matemáticamente el permiso.

CONFIGURACIÓN INICIAL

1. Dependencias:

Adicione a su archivo **build.gradle** el motor core de seguridad y de codificación basada en recursos en el cliente Oath:

```
implementation 'org.springframework.boot:spring-boot-starter-security' implementation  
'org.springframework.boot:spring-boot-starter-oauth2-resource-server'
```

2. Configuración de Propiedades **src/main/resources/application.properties**

Registraremos atributos dinámicos que le enseñarán a la aplicación el ciclo de vida y la clave secreta (HMAC) de encriptación para el Token JWT. Mantén tu clave segura y lo suficientemente larga (mínimo 256 bits / 32 caracteres).

```
jwt.expiry=5
```

```
jwt.secret=MiClaveSecretaSuperSeguraParaJWT2026EnProgramacionAvanzadaUniquindio (Adicione también
```

la información para el Seed Data de Usuarios que configuraremos mas adelante).

```
# Seed de Datos por defecto  
default-users.users[0].username=admin@solicitudes.com  
default-users.users[0].password=admin123  
default-users.users[0].role=ADMIN  
default-users.users[1].username=agente@solicitudes.com  
default-users.users[1].password=agente123
```



```
default-users.users[1].role=USER
```

LA ESTRUCTURA DE LA IDENTIDAD

3. La Entidad de Seguridad

Recuerda que hasta este momento nuestro proyecto Solicitudes no contenía conceptualmente un Agregado puro para "Usuarios Reales" operando dentro del ecosistema; simulabamos propietarioId con Strings. Para que la seguridad fluya debes crear e integrar tus credenciales a JPA.

Crear UsuarioEntity en su paquete

```
co.edu...solicitudes.infrastructure.persistence.jpa.entity :
import jakarta.persistence.*;
import lombok.*;
import java.util.UUID;

@Entity
@Table(name = "usuarios")
@Getter
@Setter
@NoArgsConstructor
public class UsuarioEntity {

    @Id
    private String id = UUID.randomUUID().toString();

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private RolSeguridadEnum rol;
}
```

Crear el Enumerador:

```
public enum RolSeguridadEnum {
    ADMIN, USER
}
```

Crear su UsuarioJpaRepository :

```
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UsuarioJpaRepository extends JpaRepository<UsuarioEntity, String> Optional<UsuarioEntity>
    findByEmail(String email);
}
```

4. Seed Data (Inyección de los Usuarios por Defecto en H2)

De la misma manera que en infraestructuras previas, configuraremos la inserción utilizando *Lombok Records* para mapear eficientemente las propiedades.

Cree un subpaquete dentro de la infraestructura llamado

co.edu.uniquindio.[proyecto].infrastructure.config.setup e inserte estas utilitarias de inicio automático.

```
import lombok.Getter;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.context.properties.bind.ConstructorBinding;
import java.util.List;

@Getter
@ConfigurationProperties(prefix = "default-users")
@RequiredArgsConstructor(onConstructor_ = @ConstructorBinding)
public class DefaultUserProperties {
    private final List<DefaultUser> users;
    // Uso impecable de la instrucción Record de Java
    public record DefaultUser(String username, String password, RolSeguridadEnum role) {}
}
```

Y ahora el inicializador para registrar con BCrypt directamente en JPA. Tenga en cuenta el uso de PasswordEncoder .

```
import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import
```

```
org.springframework.stereotype.Component;
```

```
@Component
```

```
@RequiredArgsConstructor
```

```
public class DefaultUserInitializer implements CommandLineRunner {
```

```
    private final DefaultUserProperties props;
```

```
    private final UsuarioJpaDataRepository repository;
```

```
    private final PasswordEncoder encoder;
```

```
@Override
```

```
public void run(String... args) {
```

```
    if (repository.count() == 0) {
```

```
        props.getUsers().forEach(defaultUser -> {
```

```
            UsuarioEntity eu = new UsuarioEntity();
```

```
            eu.setEmail(defaultUser.username());
```

```
            // Encriptación Nativa y Obligatoria en Spring Security
```

```
            eu.setPassword(encoder.encode(defaultUser.password()));
```

```
            eu.setRol(defaultUser.role());
```

```
            repository.save(eu);
```

```
        });
```

```
        System.out.println("✅ Security Seed Data finalizado en H2");
```

```
    }
```

```
}
```

```
}
```

Recuerda adicionar en la clase Raíz de tu App (main) la anotación atadura

```
@EnableConfigurationProperties(DefaultUserProperties.class) .
```

CEREBRO DE AUTENTICACIÓN: SPRING SECURITY

5. Clave de Firma Simétrica (HMAC)

A diferencia de modelos empresariales complejos que requieren llaveros RSA asimétricos, para esta implementación pedagógica usaremos el algoritmo **HMAC (HS256)**. HMAC es una solución simétrica que utiliza una **única cadena secreta compartida** tanto para firmar como para validar el token. Ya has definido tu clave en el application.properties (jwt.secret), pero recuerda que en un entorno de producción real, **nunca** debes quemar claves en el código; deben ir en variables de entorno y

contar con una entropía criptográfica extremadamente alta.

6. Contratos de Comunicación (DTOs de Seguridad)

En el paquete `co.edu.uniquindio.[proyecto].infrastructure.rest.dto` cree los DTOs que se usarán para transportar las credenciales durante el inicio de sesión como records:

```
import jakarta.validation.constraints.NotBlank;
public record LoginRequest(@NotBlank String username, @NotBlank String password) {}

import java.time.Instant;
import java.util.Collection;

public record TokenResponse(String token, String type, Instant expireAt, Collection<Str
```

NOTA CLAVE: A Spring Security no le interesa tu `UsuarioEntity`, sino un contrato universal de tipo `UserDetails`. En el paquete `co.edu.uniquindio.[proyecto].infrastructure.security`, envuelve Spring a tus entidades así:

```
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority; import
org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;
import java.util.List;

public record CustomUserDetails(UsuarioEntity user) implements UserDetails { @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority(user.getRol().name()));
    }

    @Override
    public String getPassword() { return user.getPassword(); }

    @Override
    public String getUsername() { return user.getEmail(); }
}
```

7. Core de Configuración de Filtros de Seguridad

En el paquete `co.edu.uniquindio.[proyecto].infrastructure.security` cree la clase `SecurityConfig`. Spring Boot 3/4 deprecia las viejas clases `WebSecurityConfigurerAdapter` a favor del control total desde una

Cadena de Filtros Declarativa (Lambda DSL).

```
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.config.annotation.authentication.configuration.Auth import
org.springframework.security.config.annotation.method.configuration.EnableMethod import
org.springframework.security.config.annotation.web.builders.HttpSecurity; import
org.springframework.security.config.annotation.web.configurers.AbstractHttpConfi import
org.springframework.security.config.http.SessionCreationPolicy; import
org.springframework.security.crypto.factory.PasswordEncoderFactories; import
org.springframework.security.crypto.password.PasswordEncoder; import
org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.web.SecurityFilterChain;
```

@Configuration

@RequiredArgsConstructor

@EnableMethodSecurity

```
public class SecurityConfig {
```

```
    private final JwtDecoder jwtDecoder;
```

@Bean

```
    public SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exc return http
        .csrf(AbstractHttpConfigurer::disable)
        // Permitir iframes para que funcione H2 Console
        .headers(headers -> headers.frameOptions(frameOptions -> frameOptions.sameO // Lógica Pura por
        Función Lambda según estándar moderno
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(
                "/api/auth/login",
                "/public/**",
                "/error",
                "/api-docs/**",
                "/swagger-ui/**",
                "/swagger-ui.html",
                "/h2-console/**"
            ).permitAll()
            .requestMatchers("/api/solicitudes/**").authenticated()
            .anyRequest().authenticated()
        )
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreation
        .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.decoder(jwtDecoder)))
```

```

        .build();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration conf return
        configuration.getAuthenticationManager();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }
}

```

NOTA CLAVE SOBRE EXCEPCIONES Y HERRAMIENTAS: Observarás que incluimos rutas como `/error` , `/api-docs/**` y `/h2-console/**` dentro del `permitAll()` . Hacer esto es vital porque: 1) Si ocurre un error enviando un JSON inválido o malas credenciales al login, Spring reenvía internamente la respuesta hacia `/error` . Si ese endpoint no está liberado, Security mascara el error y devuelve un seco Header 401 Unauthorized que frustra al programador impidiéndole ver qué falló realmente. 2) Swagger Web y H2 funcionan como recursos apartes que exigen libertad de acceso en desarrollo. Para H2 también se exige flexibilizar el bloqueo nativo de X-Frame-Options (Same Origin).

8. Enganche del Repositorio de Base de datos

En el paquete `co.edu.uniquindio.[proyecto].infrastructure.security` cree la clase `UserConfig` . Agregamos la configuración indicando explícitamente a Spring Security dónde se consiguen los usuarios para el momento del login contra nuestra base de datos.

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.UserDetailsService; import
org.springframework.security.core.userdetails.UsernameNotFoundException;

@Configuration
public class UserConfig {
    @Bean
    public UserDetailsService userDetailsServiceFromDataBase(UsuarioJpaDataRepository r return username ->
        repository.findByEmail(username)
            .map(CustomUserDetails::new)

```

```
        .orElseThrow(() -> new UsernameNotFoundException("Usuario inexistente e }  
    }  
}
```

COMPLETANDO EL PUENTE DEL TOKEN

La configuración de encriptación simétrica requiere un proveedor genérico de tokens (Encoder y Decoder).

En el paquete `co.edu.uniquindio.[proyecto].infrastructure.security` cree la clase `JwtTokenProvider`, quien se encargará de estampar las firmas (claims) y encriptar el token usando nuestro algoritmo.

```
import lombok.RequiredArgsConstructor;  
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;  
import org.springframework.security.oauth2.jwt.JwsHeader;  
import org.springframework.security.oauth2.jwt.JwtClaimsSet;  
import org.springframework.security.oauth2.jwt.JwtEncoder;  
import org.springframework.security.oauth2.jwt.JwtEncoderParameters; import  
org.springframework.stereotype.Component;  
  
import java.time.Instant;  
import java.util.Collection;  
  
@Component  
@RequiredArgsConstructor  
public class JwtTokenProvider {  
  
    private final JwtEncoder encoder;  
  
    public String generateTokenAsString(String username, Collection<String> roles, Inst JwsHeader jwsHeader =  
  
        JwsHeader.with(MacAlgorithm.HS256).build();  
  
        JwtClaimsSet claims = JwtClaimsSet.builder()  
            .issuer("solicitudes-app")  
            .issuedAt(issuedAt)  
            .expiresAt(expiresAt)  
            .subject(username)  
            .claim("roles", roles)  
            .build();
```

```

        return this.encoder.encode(JwtEncoderParameters.from(jwsHeader, claims)).getTok }
    }
}

```

A continuación, en el mismo paquete `co.edu.uniquindio.[proyecto].infrastructure.security` proveemos la configuración `JwtConfig` que leerá la clave maestra en el servidor y habilitará la lectura y escritura oficial de Spring Security de ida y vuelta.

```

import com.nimbusds.jose.jwk.source.ImmutableSecret;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;
import org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationConverter;
import org.springframework.security.oauth2.server.resource.authentication.JwtGrantedAuthoritiesConverter;

import javax.crypto.spec.SecretKeySpec;

@Configuration
public class JwtConfig {

    @Value("${jwt.secret}")
    private String jwtSecret;

    @Bean
    public JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtGrantedAuthoritiesConverter grantedAuthoritiesConverter = new JwtGrantedAuthoritiesConverter();
        grantedAuthoritiesConverter.setAuthoritiesClaimName("roles");
        grantedAuthoritiesConverter.setAuthorityPrefix("");

        JwtAuthenticationConverter authenticationConverter = new JwtAuthenticationConverter();
        authenticationConverter.setJwtGrantedAuthoritiesConverter(grantedAuthoritiesConverter);
        authenticationConverter.setJwtGrantedAuthoritiesConverter(grantedAuthoritiesConverter);
        return authenticationConverter;
    }

    @Bean
    JwtDecoder jwtDecoder() {
        return NimbusJwtDecoder.withSecretKey(new SecretKeySpec(jwtSecret.getBytes(), "HMACSHA256")).build();
    }

    @Bean
    JwtEncoder jwtEncoder() {
        return NimbusJwtEncoder.newEncoder();
    }
}

```



```

    JwtEncoder jwtEncoder() {
        return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSecret.getBytes())); }
}

```

9. Construcción del Servicio de Negocio de Seguridad (SecurityService)

En el paquete `co.edu.uniquindio.[proyecto].application.services` exponga la interfaz del servicio. Por otro lado, su implementación (`SecurityServiceImpl`) siendo un pegamento denso de infraestructura (`AuthenticationManager`), lógicamente debe residir en `co.edu.uniquindio.[proyecto].infrastructure.security.services` .

Interfaz:

```

import co.edu.uniquindio.[proyecto].dtos.LoginRequest;
import co.edu.uniquindio.[proyecto].dtos.TokenResponse;

public interface SecurityService {
    TokenResponse login(LoginRequest request);
}

```

Implementación Transaccional (SecurityServiceImpl):

```

import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken; import
org.springframework.security.core.GrantedAuthority;
import org.springframework.stereotype.Service;

import java.time.Instant;
import java.time.temporal.ChronoUnit;

@Service("securityService")
@RequiredArgsConstructor
public class SecurityServiceImpl implements SecurityService {

    // Inyectamos nuestros componentes customizados de validación de JWT private final
    JwtTokenProvider jwtTokenProvider;
    // AuthenticationManager provee la protección contra ataques de fuerza bruta al ver private final
    AuthenticationManager authenticationManager;

    @Value("${jwt.expiry}")

```

```

private long expiry;

@Override
public TokenResponse login(LoginRequest request) {

    // Dispara la búsqueda en Base de Datos que configuraste en UserConfig final var
    authentication = authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.username(), request.passwor );

    // Si no lanza excepción, el login es correcto. Procedemos a leer el Enum del R final var roles =
    authentication.getAuthorities().stream()
        .map(GrantedAuthority::getAuthority).toList();

    final var now = Instant.now();
    final var expire = now.plus(expiry, ChronoUnit.MINUTES);

    // Empaquetamos la construcción final del Token JWT
    return new TokenResponse(
        jwtTokenProvider.generateTokenAsString(authentication.getName(), roles, now "Bearer", expire, roles
    );
}
}

```

10. Construcción del Controlador de Autenticación (AuthRestController)

En el paquete co.edu.uniquindio.[proyecto].infrastructure.rest.controllers cree la clase SecurityController para exponer el endpoint de inicio de sesión.

```

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class SecurityController {

    private final SecurityService securityService; // Servicio con componente generador

```

```
@PostMapping("/login")
public ResponseEntity<TokenResponse> login(@Valid @RequestBody LoginRequest request return
    ResponseEntity.ok(securityService.login(request));
}
```

11. Restricciones Avanzadas a Nivel de Negocio (Control de Roles)

Si ya un usuario pudo llegar al Caso de Uso, pueden controlarse limitaciones internas y atómicas de qué variables puede ver según su posición de Autoridad usando

```
@PreAuthorize("hasAuthority('ADMIN')") .
```

Criterios de Evaluación Final

Configuración Lambda estructurada puramente en las nuevas especificaciones de Framework. Ausencia completa de "Security AutoConfigurations" reventando el proyecto (Control manual asertivo).

Consumo óptimo de DTOs Record.

Capacidad de login exitoso mediante Request a Base de Datos nativa y confirmada en Postman mediante Token Bearer temporal de vida media (Expira).

PARA LA PRÓXIMA CLASE: ASEGURAMIENTO DE LA CALIDAD

Los sistemas sofisticados con bases de datos transaccionales, mapeadores de inyección en tiempo real y cadenas de seguridad impenetrables son propensos al quiebre si su capa de evolución es frágil. Ahora que nuestro ciclo integral ha concluido, la siguiente labor se centra única y exclusivamente en nuestra Guía **13**. En ella profundizaremos:

Cómo lidiar con los errores generados por Spring Security ante la fase de validación de Integración.

Construcción de *Mocks* y suplantación de la red desde Controladores a Repositorios.

Implementación de `@DataJpaTest` y `@WebMvcTest` .